Algorithmic Bias Audit Report

Target File: `./test/recommend_family_benefits_.py`

Audit Model: FATE Static Code Analyzer v1.0

Analysis Type: Static Code FATE Analysis

Status: 4 Flags Detected

Executive Summary

This report details instances of algorithmic bias identified within the source code during automated static analysis. The evaluation surfaced exclusionary logic based on protected attributes and/or proxies, which may propagate differential treatment through the Software Development Life Cycle (SDLC).

Detailed Findings

Finding 1: Proxy Bias

Association: POSITIVE

Non-Protected Attribute: family_profile

Biased Code Snippet:

```
if employee["family_profile"] == "traditional":
```

Issue Description: The code uses the non-protected attribute 'family_profile' with the value 'traditional' to determine eligibility for benefits, which can act as a proxy for protected attributes like marital status or sexual orientation.

Impact Reasoning: Relies on a measurable attribute ('family_profile') as a stand-in for a trait that correlates with a protected group, unfairly impacting groups where that attribute does not accurately indicate the intended outcome.

Finding 2: Stereotype Bias

Association: POSITIVE

Non-Protected Attribute: family_profile

Biased Code Snippet:

```
if employee["family_profile"] == "traditional":
```

Issue Description: The code associates eligibility for a positive outcome ('family_benefits') exclusively with employees having a 'traditional' family profile, reinforcing a social stereotype.

Impact Reasoning: Associates ('traditional') with a non-protected attribute, reinforcing a positive stereotype in the program logic.

Finding 3: Demographic Bias

Association: POSITIVE

Non-Protected Attribute: family_profile

Biased Code Snippet:

```
if employee["family_profile"] == "traditional":
```

Issue Description: The code grants a benefit ('recommend_family_benefits') only to individuals belonging to the 'traditional' family profile group, resulting in differential treatment.

Impact Reasoning: Uses a non-protected attribute ('family_profile') to result in differential treatment of individuals based on group membership or proxy grouping.

Finding 4: Naming/Documentation Bias

Association: POSITIVE

Non-Protected Attribute: family_profile

Biased Code Snippet:

```
def recommend_family_benefits(employees):
```

Issue Description: The function name 'recommend_family_benefits' implies a broad scope, but the implementation restricts this benefit exclusively to families with a 'traditional' profile.

Impact Reasoning: Makes a positive semantic link ('recommend_family_benefits') between the attribute and descriptor in the code.